

Studio di Robustezza al Rumore — MAGIC Gamma Telescope

Analisi dei risultati, dei grafici e delle tabelle

Progetto «Architetture Dati» — Daniele Maccagnan

Questo documento legge e interpreta **tutti** i risultati prodotti dal notebook **progettoArchicopy**: tabelle, curve, confusion matrix, grafici di confronto e di sintesi. L'obiettivo non è ripetere i numeri ma **spiegare perché vengono fuori quei numeri**, mettendo in relazione il *meccanismo* di ogni rumore con l'*architettura* di ogni modello, e collegando le sezioni tra loro.

0. Risultato in una riga (TL;DR)

Tre modelli — **NN Base** (rete senza regolarizzazione), **NN Pro** (stessa rete + dropout + L2 + class-weight + early stopping) e **Random Forest** (200 alberi) — sono stati stressati con **6 tipi di rumore a 3 intensità** (10%, 25%, 40%), sporcando **solo il training** (validation e test restano puliti, tranne i *missing values* che vengono sporcati ovunque e poi imputati).

Il quadro che emerge è netto e si riassume così:

	Vince quando...	Perché
NN Pro	il segnale è corrotto ma presente → feature noise, label noise (random e borderline)	regolarizzazione = freno alla memorizzazione del rumore
Random Forest	l'integrità dei dati è compromessa → outlier, missing values, missing feature	split robusti a valori estremi + ridondanza fra feature
NN Base	quasi mai: è il riferimento «ingenuo» che mostra cosa succede <i>senza</i> difese	overfitting visibile sulle etichette sporche

Gradiente di pericolosità dei rumori (dal più innocuo al più devastante), valido per tutti i modelli:

Outlier « Feature noise < Missing values < Missing feature ≈ Label noise random « Label noise borderline

Ranking di robustezza complessiva (accuracy media sui 6 rumori al 40%):

Modello	Accuracy media @40%
NN Pro	0.7667
RF	0.7626
NN Base	0.7576

Le tre medie sono vicinissime (entro ~0.9 punti): **non esiste un vincitore unico**, la storia vera è la *specializzazione* modello × rumore descritta sopra.

1. Setup essenziale (per leggere i numeri)

Dataset. MAGIC Gamma Telescope: 19 020 eventi, 10 feature continue (parametri geometrici dell'immagine Cherenkov) + 1 etichetta binaria **class** = gamma (1) / hadron (0). Dopo la rimozione di 41 duplicati, il training conta 11 371 righe. Split **60/20/20** stratificato → Train 11 412 · Val 3 804 · Test 3 804.

Sbilanciamento. La classe è ~**64.8% gamma / 35.2% hadron**. È un dettaglio che torna in continuazione: quando un modello degrada, lo fa quasi sempre **sacrificando la classe minoritaria (hadron)**, cioè crollando sul *recall della classe 0* e rifugiandosi sulla maggioranza. Lo si vedrà in ogni tabella.

I tre modelli.

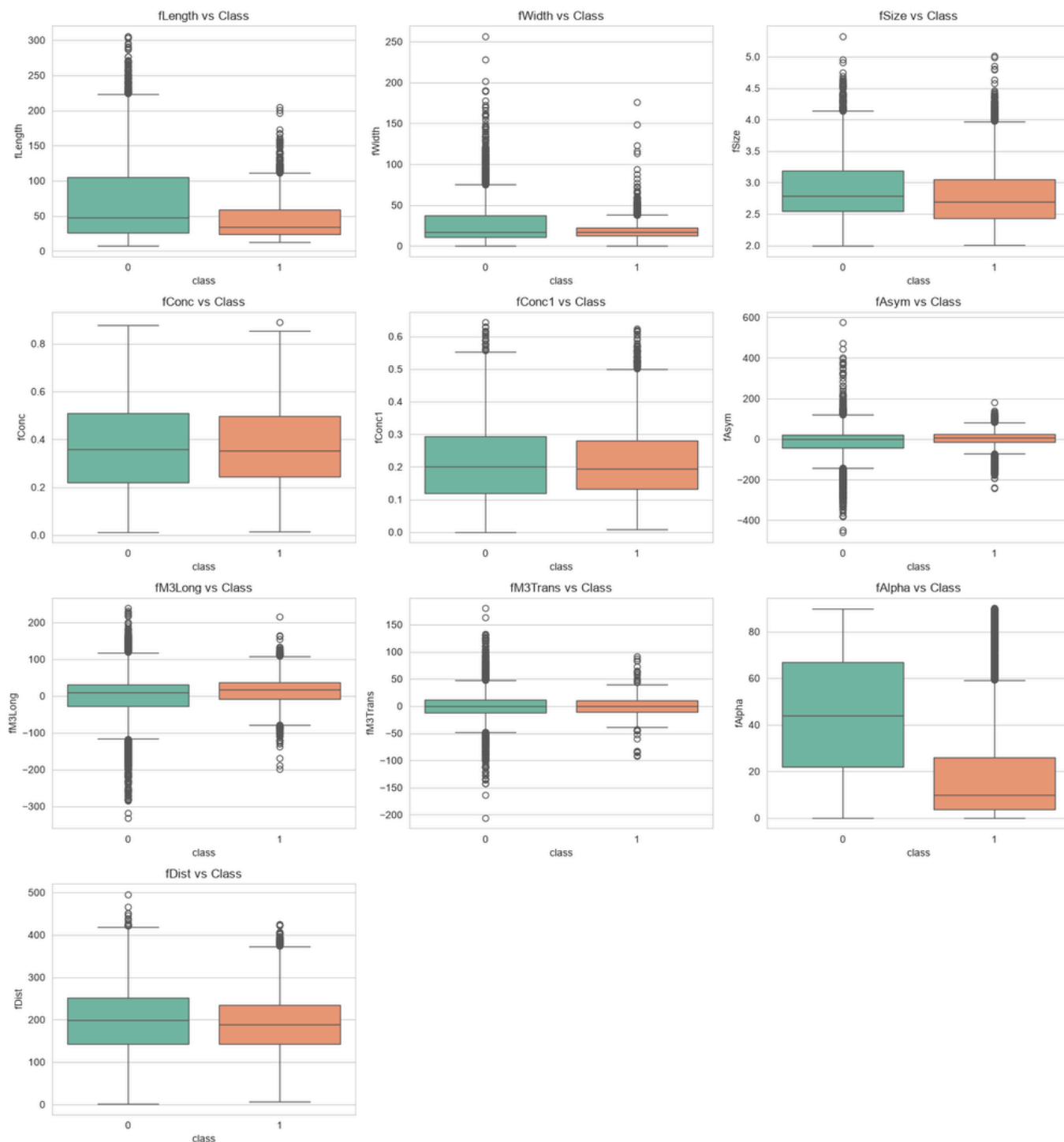
1. **NN Base** — **Dense(32) → Dense(16) → sigmoid**, Adam, **nessuna** regolarizzazione, 75 epoche fisse. È la cavia: senza freni, *fa quello che i dati le dicono di fare*, rumore incluso.
2. **NN Pro** — stessa architettura + **Dropout(0.2) + L2(1e-4) + class_weight bilanciato + EarlyStopping(patience 15, restore best)**. Ogni aggiunta è un freno alla memorizzazione: il dropout impedisce di affidarsi a singoli neuroni, l'L2 tiene i pesi piccoli, l'early stopping ferma l'addestramento **prima** che la rete inizi a inseguire il rumore.
3. **Random Forest** — 200 alberi, profondità libera. Robusto per costruzione a feature ridondanti e a valori estremi (gli split usano l'*ordinamento*, non la magnitudine), ma con alberi non potati è **vulnerabile alla memorizzazione delle etichette sbagliate**.

Metriche. Accuracy, AUC (capacità di ranking, indipendente dalla soglia), F1 macro (media non pesata sulle due classi → penalizza i crolli sulla minoritaria) e recall per classe.

2. Perché i risultati sono quelli che sono — le 4 cose dell'EDA che spiegano (quasi) tutto

Prima di guardare il rumore conviene fissare quattro fatti emersi dall'analisi esplorativa: sono le **cause** di buona parte dei comportamenti successivi.

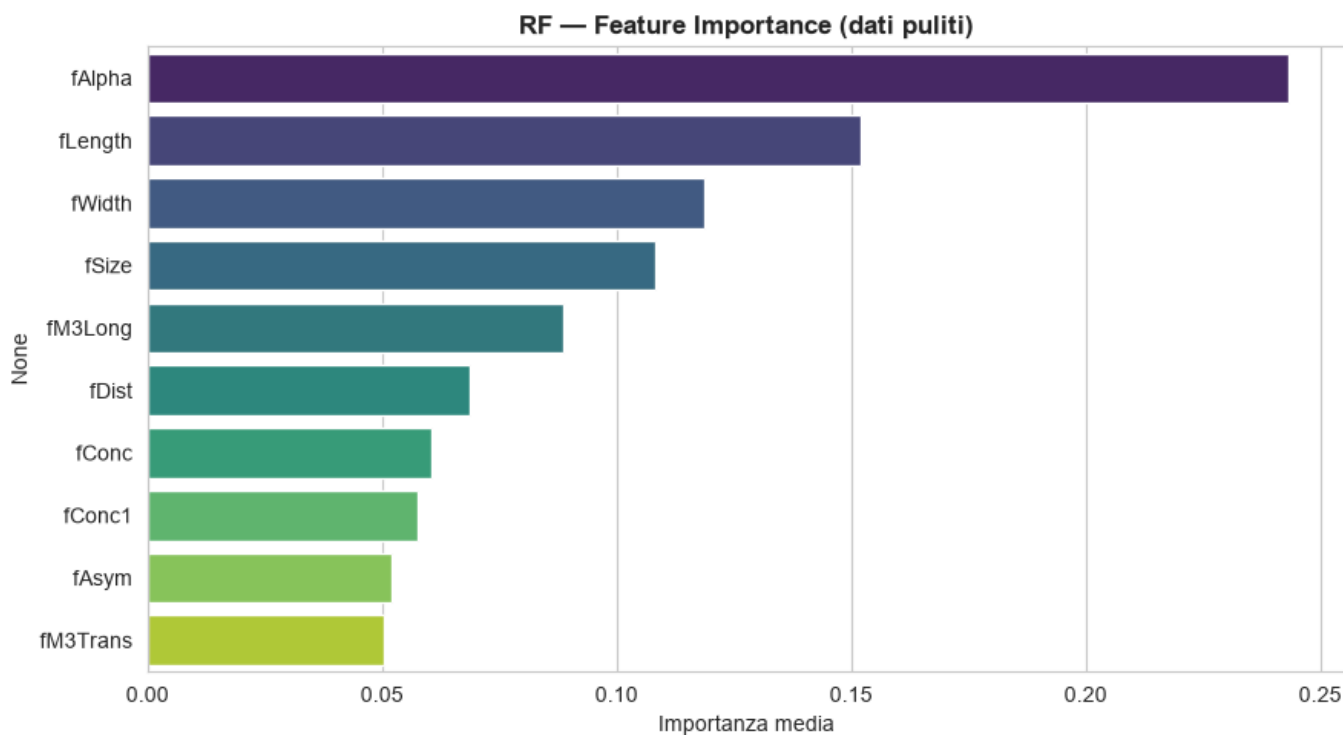
2.1 Una feature domina tutte le altre: **fAlpha**



Boxplot di ogni feature separata per classe (0 = hadron, 1 = gamma): *fAlpha* e la sola feature con separazione netta fra le due classi.

I boxplot per classe dicono una cosa sola, fortissima: **fAlpha** è l'unica feature che separa davvero gamma da hadron. Per gli hadron (classe 0) la mediana è ~44° con grande dispersione; per i gamma (classe 1) è schiacciata verso ~10°, vicino a 0. È fisica: i gamma puntano alla sorgente (angolo piccolo), gli adroni arrivano isotropi. Tutte le altre feature (*fConc*, *fConc1*, *fDist*, *fAsym*, *fM3**) hanno box quasi sovrapposti fra le due classi → da sole discriminano poco. *fLength* e *fWidth* danno un contributo modesto.

È la conferma quantitativa nella feature importance del Random Forest:



Importanza media delle feature secondo il Random Forest: *fAlpha* vale da sola circa quanto le due successive insieme.

fAlpha ≈ 0.24 , quasi il doppio della seconda (*fLength* ≈ 0.15). Da qui discende **tutta** la sezione *Missing Feature*: i livelli di quel rumore rimuovono prima *fAlpha* (10%), poi *fAlpha+fLength+fWidth* (25%), poi anche *fSize+fM3Long* (40%). Togliere la variabile più informativa *per prima* è il motivo per cui quel rumore fa così male e così in fretta.

2.2 Ridondanza vs. indipendenza: la matrice di correlazione

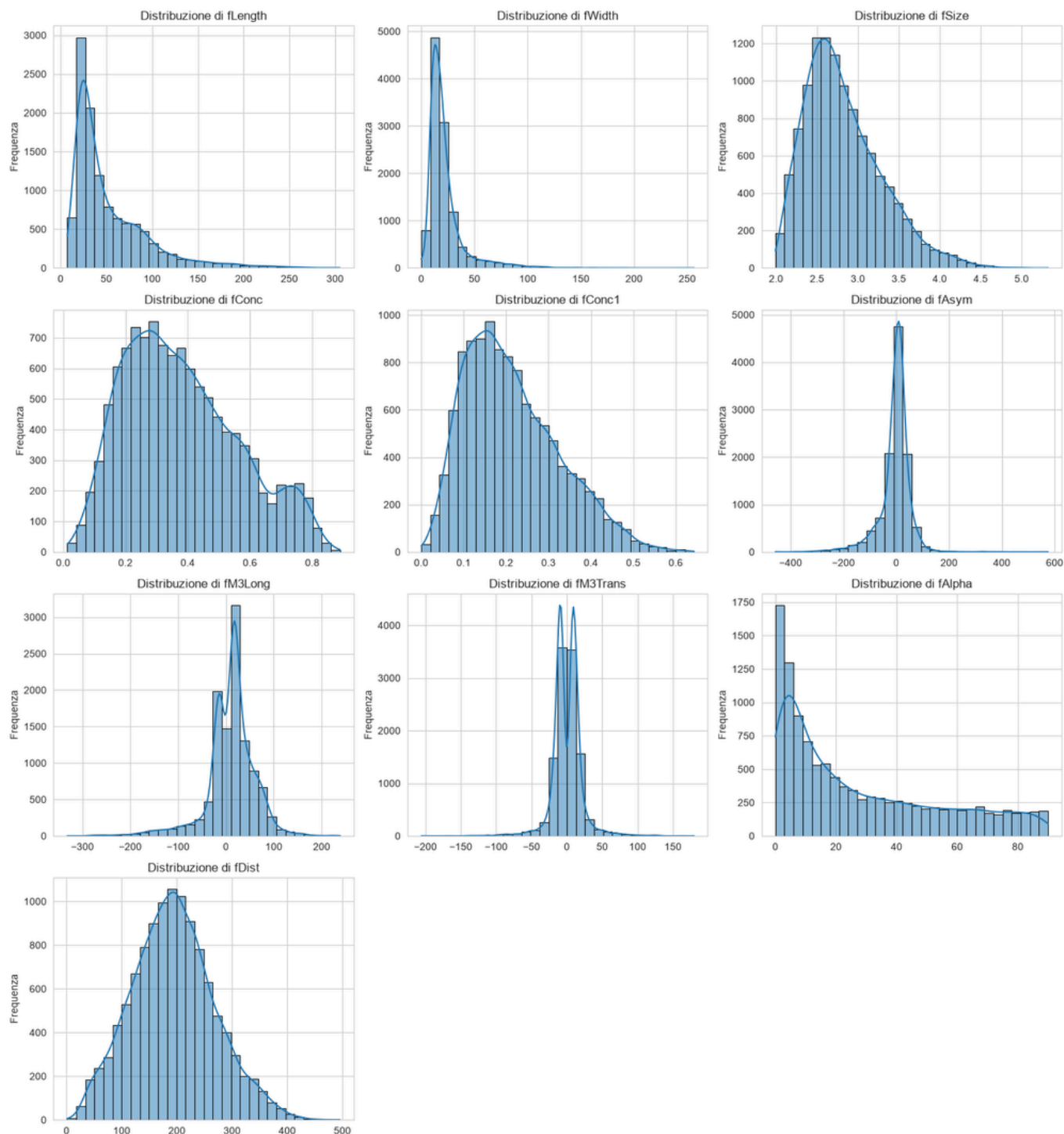


Matrice di correlazione fra le 10 feature: *fConc*-*fConc1* quasi identiche (0.98); *fAlpha* scorrelata da tutto.

Due gruppi saltano all'occhio:

- **Forte ridondanza** — *fConc*/*fConc1* sono **quasi la stessa variabile** ($\rho = 0.98$); *fLength*/*fWidth*/*fSize* sono molto correlate (0.71–0.77), e *fSize* è anti-correlata con *fConc* (–0.85). Quando un'informazione è *replicata* su più colonne, sporcarne o cancellarne una fa poco danno: le altre **compensano**. È la ragione per cui *feature noise* e *missing values* restano relativamente gestibili.
- **Indipendenza** — *fAlpha* è **scorrelata da tutto** (da –0.01 a 0.23). Porta informazione *unica*: nessun'altra feature può sostituirla. Ridondanza + indipendenza spiegano insieme l'apparente paradosso che vedremo: il sistema tollera bene il rumore *diffuso* sulle feature, ma è fragilissimo quando perde *proprio fAlpha*.

2.3 Distribuzioni asimmetriche → la mediana è più sicura della media



Istogrammi delle 10 feature: quasi tutte fortemente asimmetriche o a code pesanti.

Quasi tutte le feature sono **asimmetriche** o **a code pesanti**: **fLength**, **fWidth**, **fSize**, **fAlpha** sono right-skewed; **fAsym**, **fM3Long**, **fM3Trans** sono picchi stretti su 0 con code lunghe; solo **fDist** è quasi normale. Conseguenza diretta nella sezione *Missing Values*: la **mediana** è un centro più robusto della media su distribuzioni storte (la media viene tirata dalle code) → l'imputazione con mediana batte sistematicamente quella con media. Stessa logica per gli *outlier*: la **winsorizzazione** (taglio delle code) recupera bene perché i valori estremi iniettati cadono proprio sulle code che i modelli già tolleravano poco.

2.4 Lo sbilanciamento dà il «verso» a ogni crollo

Con 64.8% di gamma, la scorciatoia più comoda per un modello in difficoltà è **predire gamma più spesso**. Per questo, ogni volta che il rumore morde, il **recall sulla classe 0 (hadron)** crolla mentre il recall sulla 1 (gamma) sale anche sopra 0.95: il modello non «sbaglia a caso», *scivola verso la maggioranza*. Il **class_weight** della NN Pro serve a contrastare proprio questo.

3. Baseline su dati puliti (il riferimento, livello 0%)

Modello	Accuracy	AUC	F1 macro	Recall hadron (0)	Recall gamma (1)
NN Base	0.8678	0.9214	0.8517	0.77	0.92
NN Pro	0.8601	0.9238	0.8468	0.80	0.89
RF	0.8785	0.9300	0.8629	0.77	0.94

Su dati puliti **RF è il più accurato** e ha l'AUC migliore; le due reti sono a un soffio. Si nota già qui la firma dello sbilanciamento: il recall sulla classe gamma (0.92–0.94) è molto più alto di quello sulla classe hadron (0.77–0.80). La NN Pro, grazie al **class_weight**, è l'unica a partire con un recall hadron a 0.80: paga ~0.8 punti di accuracy ma è **più equilibrata** fra le classi — vantaggio che diventerà decisivo sotto rumore.

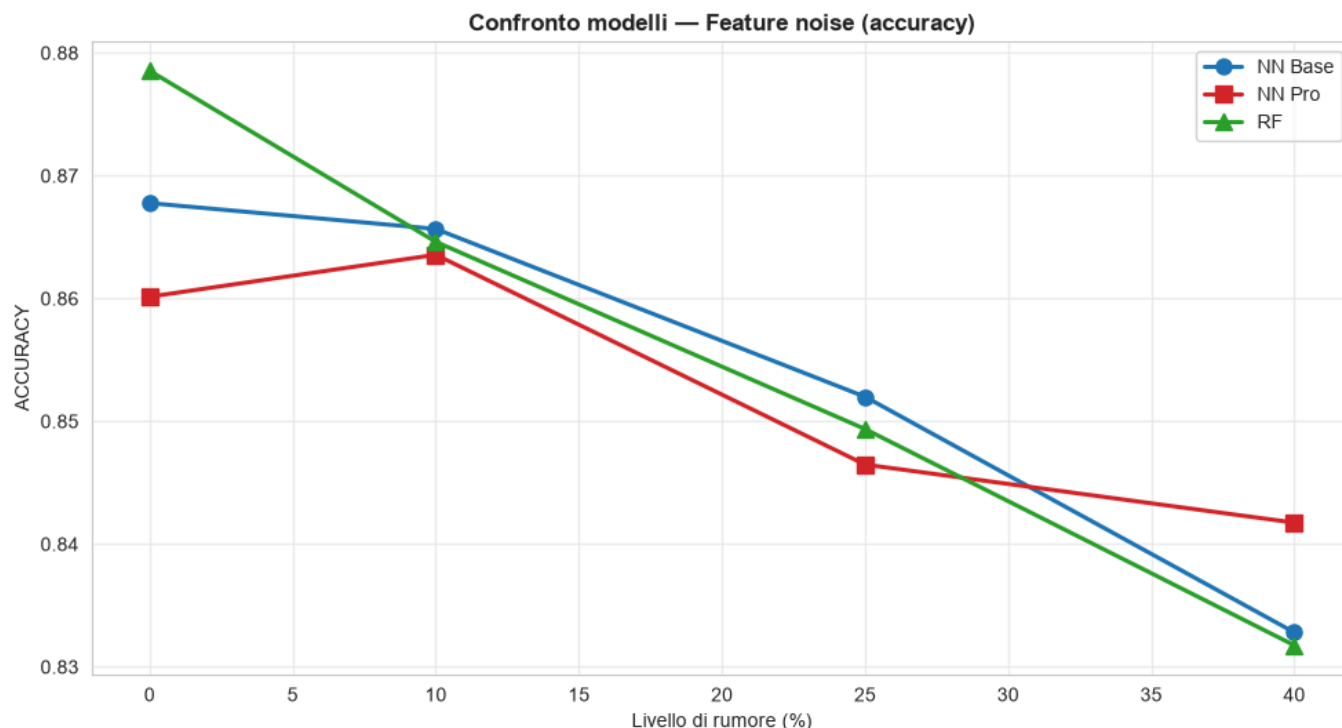
4. Risultati per tipo di rumore

Le sezioni seguono l'ordine del notebook (A–F). Per ognuna: cosa fa il rumore, i numeri, la lettura della figura e il **perché** dell'interazione con ciascun modello.

4.A — Feature Noise (rumore gaussiano sulle feature) · *degrado lieve e graduale*

Cosa fa. Aggiunge rumore gaussiano $N(0, \sigma)$ alle feature del training ($\sigma = 0.10/0.25/0.40$ in spazio standardizzato). Simula sensori imprecisi.

Modello	Metrica	0%	10%	25%	40%	$\Delta(0 \rightarrow 40)$
NN base	Accuracy	0.8678	0.8657	0.8520	0.8328	-0.0350
	AUC	0.9214	0.9187	0.9074	0.8941	-0.0273
	Recall hadron (0)	0.77	0.75	0.68	0.59	-0.18
NN pro	Accuracy	0.8601	0.8636	0.8465	0.8417	-0.0184
	AUC	0.9238	0.9204	0.9071	0.8940	-0.0298
	Recall hadron (0)	0.80	0.79	0.76	0.73	-0.07
RF	Accuracy	0.8785	0.8646	0.8494	0.8318	-0.0467
	AUC	0.9300	0.9222	0.9041	0.8913	-0.0387
	Recall hadron (0)	0.77	0.72	0.66	0.60	-0.17



Confronto dei tre modelli su accuracy al crescere del feature noise: degrado dolce e quasi parallelo.

Lettura. Calo **dolce e quasi lineare** per tutti: a 40% si perdono solo 2–5 punti di accuracy, l'AUC resta ~0.89. La **NN Pro è la più piatta** ($\Delta\text{acc} -0.018$), il **RF il più sensibile** ($\Delta\text{acc} -0.047$, $\Delta\text{auc} -0.039$).

Perché. Il rumore gaussiano è *diffuso e a media nulla*: parte si **autocancella** quando il modello aggrega molti esempi e molte feature (qui aiuta la ridondanza vista in §2.2). La NN Pro regge meglio perché L2 + dropout la abitua a non fidarsi di una singola coordinata perturbata. Il RF soffre un filo di più perché i suoi split sono *soglie su singole feature*: aggiungere rumore sposta i punti attorno alle soglie e «sfuoca» i tagli ottimali. Si vede anche la firma dello sbilanciamento: per NN Base il recall hadron scende 0.77 → **0.59** mentre quello gamma sale 0.92 → 0.97 — il modello, in dubbio, vira sulla maggioranza.

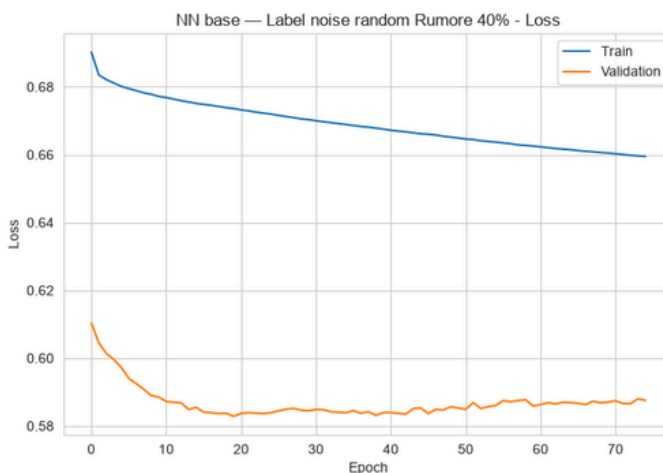
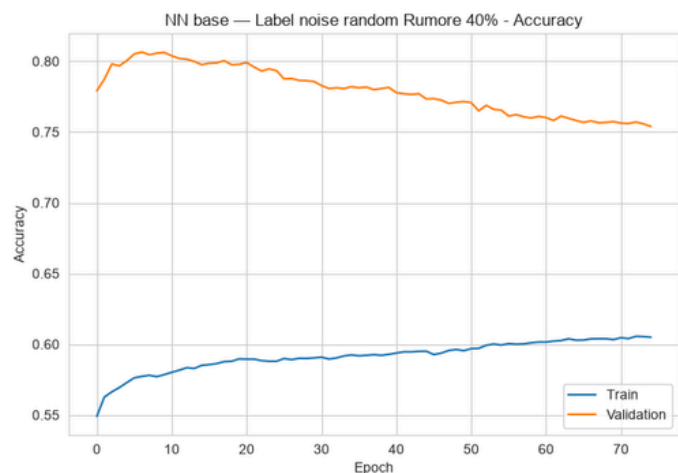
Il prezzo del feature noise è visibile anche sulle **curve ROC** (commentate in §5.5): le tre curve pulite (AUC 0.921–0.930) scivolano tutte verso ~0.89 al 40%, restando però lontanissime dalla diagonale: il rumore *peggiora* la discriminazione, non la *distrugge*.

4.B — Label Noise Random (etichette invertite a caso) · *qui le difese contano*

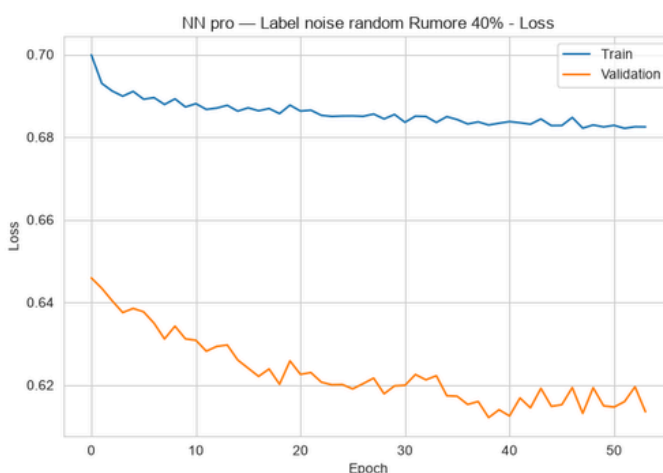
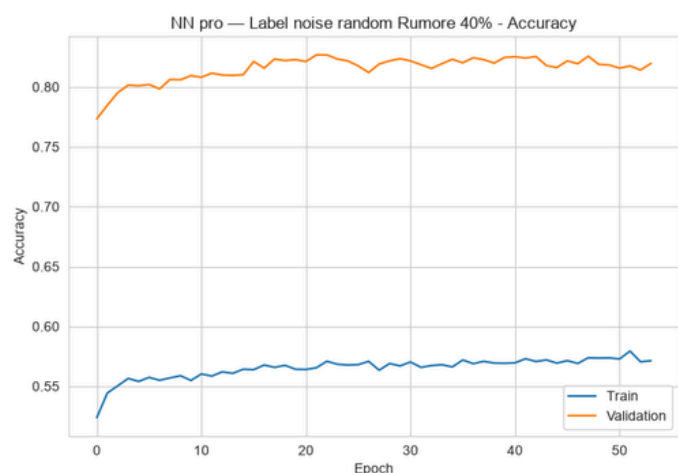
Cosa fa. Inverte casualmente il 10/25/40% delle etichette del training. Errori di annotazione uniformi.

Modello	Metrica	0%	10%	25%	40%	$\Delta(0 \rightarrow 40)$
NN base	Accuracy	0.8678	0.8623	0.8410	0.7419	-0.1259
	AUC	0.9214	0.9128	0.8882	0.7513	-0.1701
	Recall hadron (0)	0.77	0.74	0.73	0.47	-0.30
NN pro	Accuracy	0.8601	0.8570	0.8378	0.8044	-0.0557
	AUC	0.9238	0.9200	0.9081	0.8677	-0.0561

Modello	Metrica	0%	10%	25%	40%	$\Delta(0 \rightarrow 40)$
	Recall hadron (0)	0.80	0.80	0.81	0.73	-0.07
RF	Accuracy	0.8785	0.8767	0.8399	0.7179	-0.1606
	AUC	0.9300	0.9214	0.8884	0.7673	-0.1627
	Recall hadron (0)	0.77	0.76	0.73	0.66	-0.11



NN Base @40% label noise random: la validation sale, tocca il picco e poi RIDISCE (memorizzazione del rumore).



NN Pro @40% label noise random: la validation sale e RESTA alta; regolarizzazione + early stopping bloccano il degrado.

Lettura — è il confronto più istruttivo del progetto. Fino al 10% l'effetto è minimo (RF praticamente invariato a 0.877). Ma al **40%** i modelli divergono nettamente:

- **NN Base 0.7419** — la curva di validation (prima figura) **tocca il picco ~0.805 verso l'epoca 7 e poi ridiscende fino a ~0.755**: è la firma da manuale dell'**overfitting al rumore**. Senza early stopping la rete continua per 75 epoche a inseguire le etichette sbagliate e *generalizza sempre peggio*.
- **NN Pro 0.8044** — la validation (seconda figura) **sale e resta in plateau a ~0.82**, senza ridiscendere. L'accuracy di *training* resta bassa (~0.57): dropout + L2 **impediscono fisicamente**

alla rete di adattarsi ai singoli punti corrotti, e l'early stopping la ferma sul punto migliore. Risultato: **+6 punti** netti sulla NN Base.

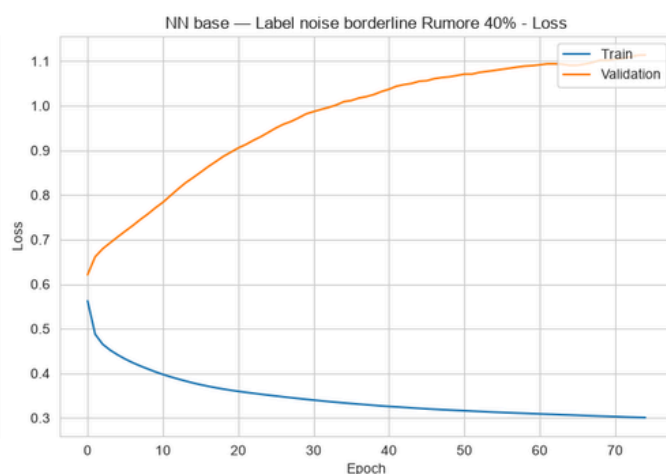
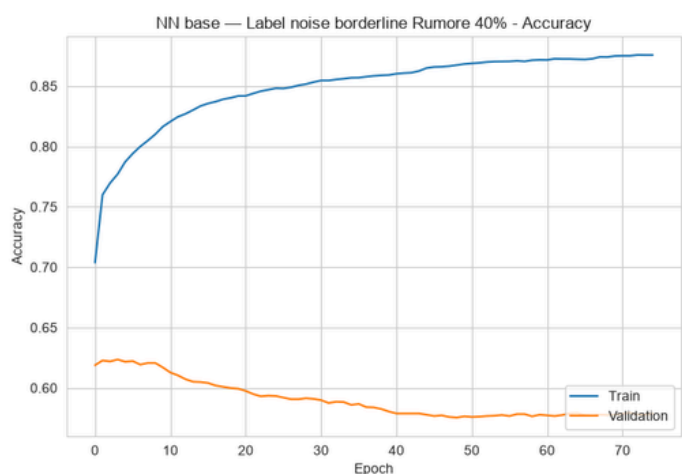
- **RF 0.7179** — sorpresa **contro-intuitiva**: il RF è il **peggiore** sotto label noise forte. I 200 alberi a profondità libera *memorizzano* i punti con etichetta sbagliata (ogni albero li isola con split profondi) e il bagging non basta a cancellare una corruzione al 40%.

Perché. Il rumore *sulle etichette* è diverso da quello sulle feature: non lo puoi mediare via, perché cambia il *bersaglio*. Chi ha un freno alla memorizzazione (NN Pro) vince; chi memorizza senza ritegno (NN Base a fine training, RF non potato) perde. **Nota importante:** la sapienza comune «il RF è robusto al label noise» vale **solo a basso rumore** (a 10% è infatti il migliore). Al 40% si ribalta. Questo *crossover* è uno dei risultati più interessanti dello studio.

4.C — Label Noise Borderline (etichette invertite sui campioni ambigui) · *il rumore più devastante*

Cosa fa. Inverte le etichette dei campioni **più vicini al centroide della classe opposta**, cioè quelli a ridosso del confine decisionale. È un rumore *mirato e coerente*, non casuale.

Modello	Metrica	0%	10%	25%	40%	$\Delta(0 \rightarrow 40)$
NN base	Accuracy	0.8678	0.8252	0.7326	0.6002	-0.2676
	AUC	0.9214	0.8680	0.7879	0.6656	-0.2558
	Recall hadron (0)	0.77	0.65	0.64	0.62	-0.15
NN pro	Accuracy	0.8601	0.8352	0.7403	0.6354	-0.2247
	AUC	0.9238	0.8915	0.8265	0.7084	-0.2154
	Recall hadron (0)	0.80	0.75	0.72	0.62	-0.18
RF	Accuracy	0.8785	0.8383	0.7387	0.6094	-0.2691
	AUC	0.9300	0.8886	0.8043	0.6774	-0.2526
	Recall hadron (0)	0.77	0.67	0.65	0.66	-0.11



NN Base @40% label noise borderline: train accuracy sale a 0.88 ma la validation loss ESPLODE (0.6 -> 1.1): confine appreso con sicurezza ma sbagliato.

Lettura. È il **rumore peggiore in assoluto**: al 40% l'accuracy crolla a **0.60–0.64** per *tutti* i modelli ($-0.22 \div -0.27$). Confronto diretto con il random a parità di %: NN Base 0.6002 (borderline) vs 0.7419 (random) → **~14 punti in meno**.

La curva è rivelatrice e **opposta** a quella del random: qui la **train accuracy sale fino a 0.88**, mentre la **validation loss esplode da 0.6 a 1.1**. Cioè il modello fitta benissimo il training... imparando un confine **sbagliato ma coerente**, e diventa *sempre più sicuro* dell'errore (per questo la loss su dati puliti schizza).

Perché borderline » random. Il random sparge flip *incoerenti*: il modello non riesce a impararli sistematicamente (train accuracy bloccata ~0.60) e in parte li media via. Il borderline invece **ridisegna un'intera regione contigua** attorno al confine: relabellando i punti vicini al centroide opposto, *sposta il confine apparente* verso l'interno della classe. Il modello impara questo confine spostato come se fosse vero. Anche la NN Pro qui aiuta poco (0.6354, comunque la «meno peggio»): la regolarizzazione frena la memorizzazione dei *singoli* punti, ma non può recuperare un confine **strutturalmente** falsato dai dati.

Morale: non tutto il label noise è uguale — *dove sbagli conta più di quanto sbagli*.

4.D — Missing Values (celle mancanti → imputazione) · *contenibile, e la mediana vince*

Cosa fa. Inserisce NaN nel 10/25/40% delle celle (di train, val e test). Keras e RF non accettano NaN: serve **imputazione**, fittata solo sul train. Si confrontano **media** vs **mediana**. Variante canonica per l'aggregato = **mediana**.

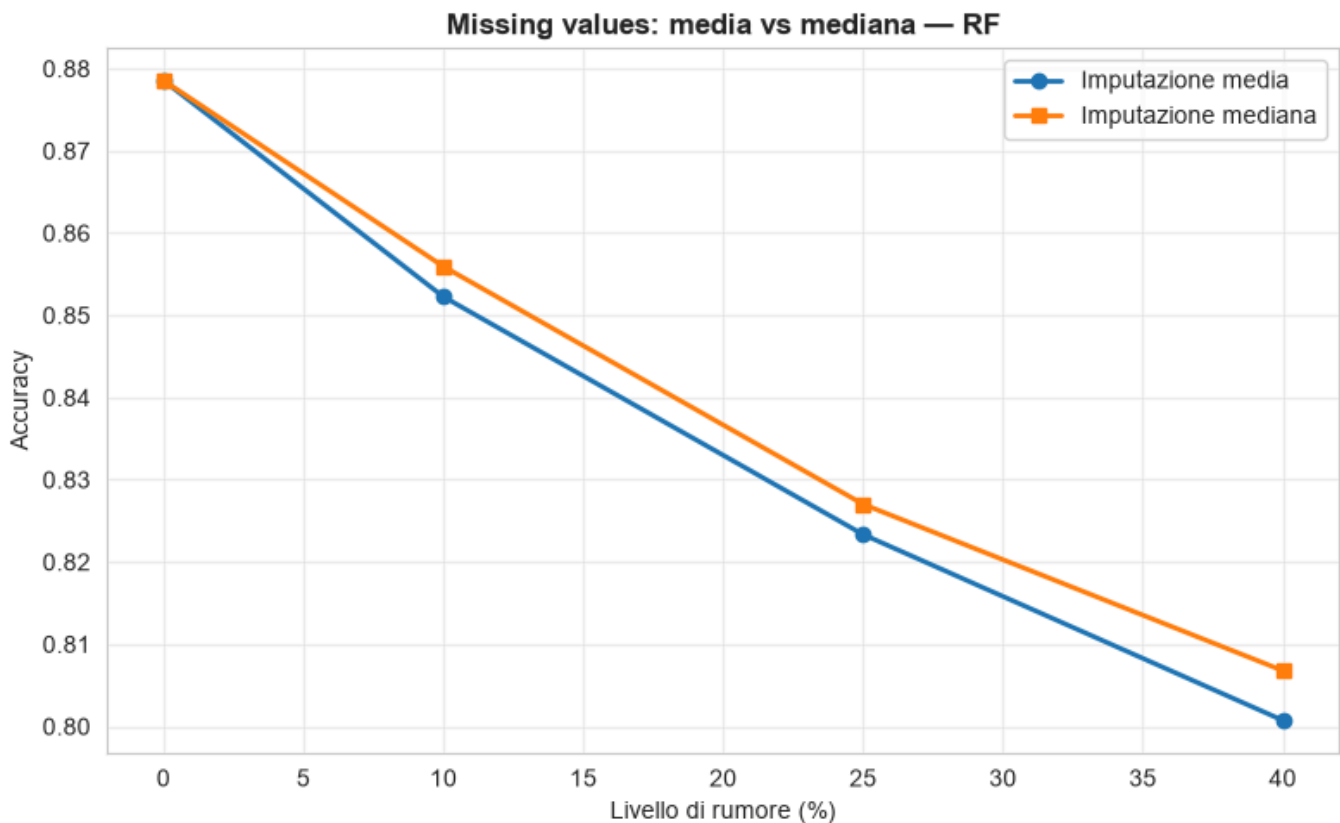
Numeri (mediana):

Modello	Metrica	0%	10%	25%	40%	$\Delta(0 \rightarrow 40)$
NN base	Accuracy	0.8678	0.8423	0.8123	0.7792	-0.0886
	AUC	0.9214	0.8947	0.8645	0.8266	-0.0948
	Recall hadron (0)	0.77	0.72	0.67	0.56	-0.21
NN pro	Accuracy	0.8601	0.8375	0.7978	0.7650	-0.0951
	AUC	0.9238	0.9002	0.8649	0.8319	-0.0919
	Recall hadron (0)	0.80	0.79	0.74	0.70	-0.10
RF	Accuracy	0.8785	0.8559	0.8270	0.8068	-0.0717
	AUC	0.9300	0.9051	0.8750	0.8477	-0.0823
	Recall hadron (0)	0.77	0.72	0.65	0.61	-0.16

Confronto **media vs mediana** (accuracy):

Modello	Strategia	0%	10%	25%	40%
NN base	Media	0.8678	0.8344	0.8023	0.7713

Modello	Strategia	0%	10%	25%	40%
NN base	Mediana	0.8678	0.8423	0.8123	0.7792
NN pro	Media	0.8601	0.8241	0.7918	0.7547
NN pro	Mediana	0.8601	0.8375	0.7978	0.7650
RF	Media	0.8785	0.8523	0.8233	0.8007
RF	Mediana	0.8785	0.8559	0.8270	0.8068



Missing values, RF: la mediana (arancio) sta sempre sopra la media (blu), con divario che cresce col rumore.

Lettura. Degrado **moderato** ($\Delta_{acc} -0.07 \div -0.10$ al 40%): l'imputazione contiene bene il danno fino al 25%. La **mediana batte la media** in ogni cella della tabella (RF @40%: 0.8068 vs 0.8007; NN Pro 0.7650 vs 0.7547), con un divario che si allarga col rumore. Il **RF è il più robusto** al 40% (0.8068).

Perché. Ogni NaN imputato è un valore «inventato» che **avvicina il punto alla media/mediana** della feature → riduce la varianza informativa e attenua proprio gli estremi che **fAlpha** usa per discriminare. La mediana vince per il motivo di §2.3 (feature asimmetriche: la media è distorta dalle code). Il RF regge meglio perché può **ridistribuire le decisioni su altre feature** (ridondanza di §2.2) quando una è stata «appiattita» dall'imputazione, mentre le reti propagano l'errore di imputazione attraverso tutti i pesi.

4.E — Outlier (valori estremi $\pm 5\sigma$) · il rumore più innocuo

Cosa fa. Sostituisce una frazione di campioni con valori estremi ($\pm 5\sigma$). Tre strategie: **lasciati**, **winsorizzazione** (taglio code), **imputazione** ($|z| > 4 \rightarrow \text{NaN} \rightarrow \text{mediana}$). Canonica per l'aggregato =

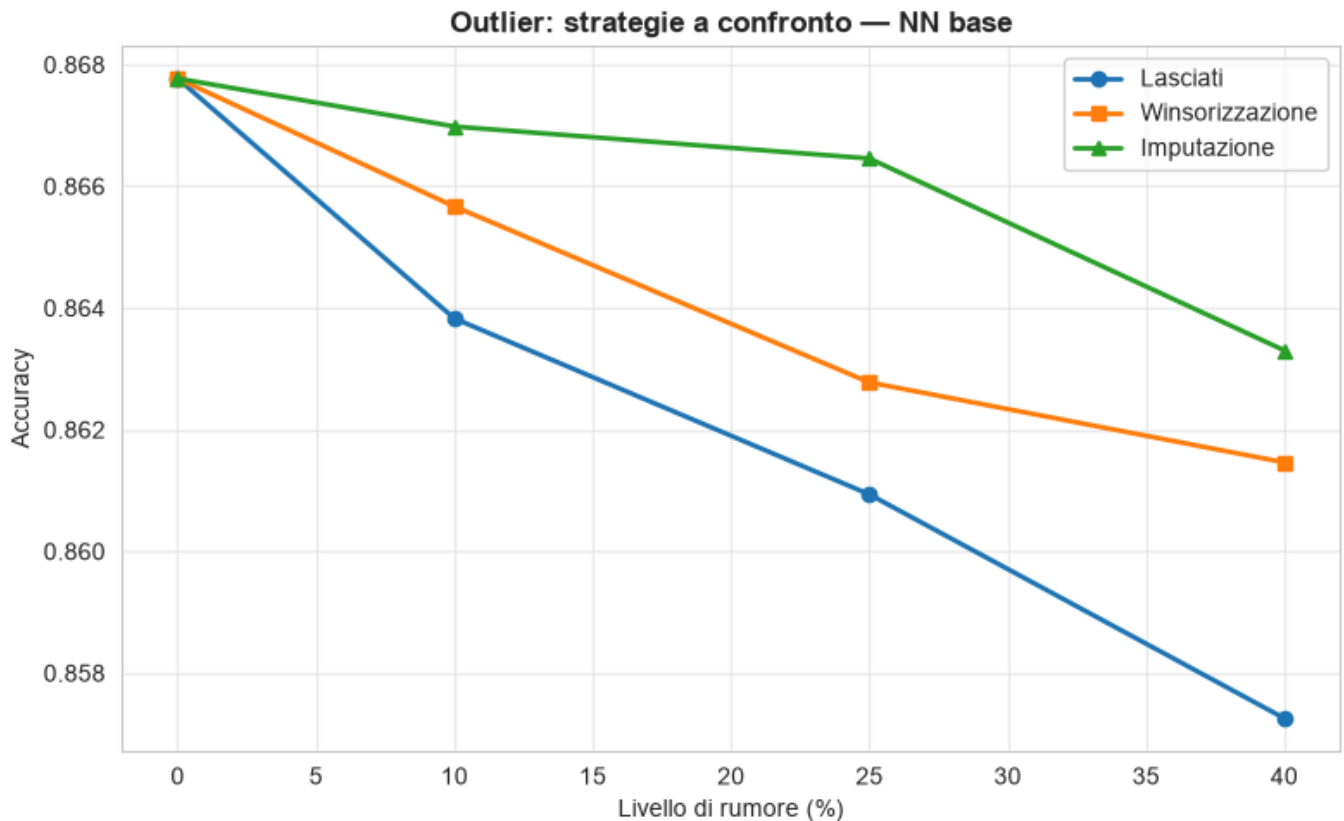
winsorizzazione.

Numeri (winsorizzazione):

Modello	Metrica	0%	10%	25%	40%	$\Delta(0 \rightarrow 40)$
NN base	Accuracy	0.8678	0.8657	0.8628	0.8615	-0.0063
	AUC	0.9214	0.9196	0.9160	0.9116	-0.0098
	Recall hadron (0)	0.77	0.74	0.74	0.75	-0.02
NN pro	Accuracy	0.8601	0.8583	0.8441	0.8541	-0.0060
	AUC	0.9238	0.9220	0.9173	0.9201	-0.0037
	Recall hadron (0)	0.80	0.80	0.81	0.79	-0.01
RF	Accuracy	0.8785	0.8788	0.8751	0.8736	-0.0049
	AUC	0.9300	0.9295	0.9267	0.9250	-0.0050
	Recall hadron (0)	0.77	0.76	0.75	0.75	-0.02

Confronto delle **tre strategie** (accuracy):

Modello	Strategia	0%	10%	25%	40%
NN base	Lasciati	0.8678	0.8638	0.8609	0.8573
NN base	Winsorizzazione	0.8678	0.8657	0.8628	0.8615
NN base	Imputazione	0.8678	0.8670	0.8665	0.8633
NN pro	Lasciati	0.8601	0.8533	0.8591	0.8509
NN pro	Winsorizzazione	0.8601	0.8583	0.8441	0.8541
NN pro	Imputazione	0.8601	0.8591	0.8604	0.8630
RF	Lasciati	0.8785	0.8796	0.8783	0.8725
RF	Winsorizzazione	0.8785	0.8788	0.8751	0.8736
RF	Imputazione	0.8785	0.8783	0.8738	0.8738



Outlier, NN Base, tre strategie (attenzione: asse y molto zoomato, tutte le differenze stanno in circa 1 punto).

Lettura. È il rumore **meno dannoso di tutti**: anche *lasciando* gli outlier l'accuracy cala di ~1 punto soltanto (Δacc tra -0.005 e -0.011); winsorizzazione e imputazione recuperano quasi tutto. Attenzione: nel grafico l'asse y è zoomatissimo (0.857–0.868), quindi le «differenze» visive corrispondono a frazioni di punto. Il **RF è di fatto immune** ($\approx$ invariato), e tra le strategie imputazione/winsorizzazione fanno meglio del lasciare.

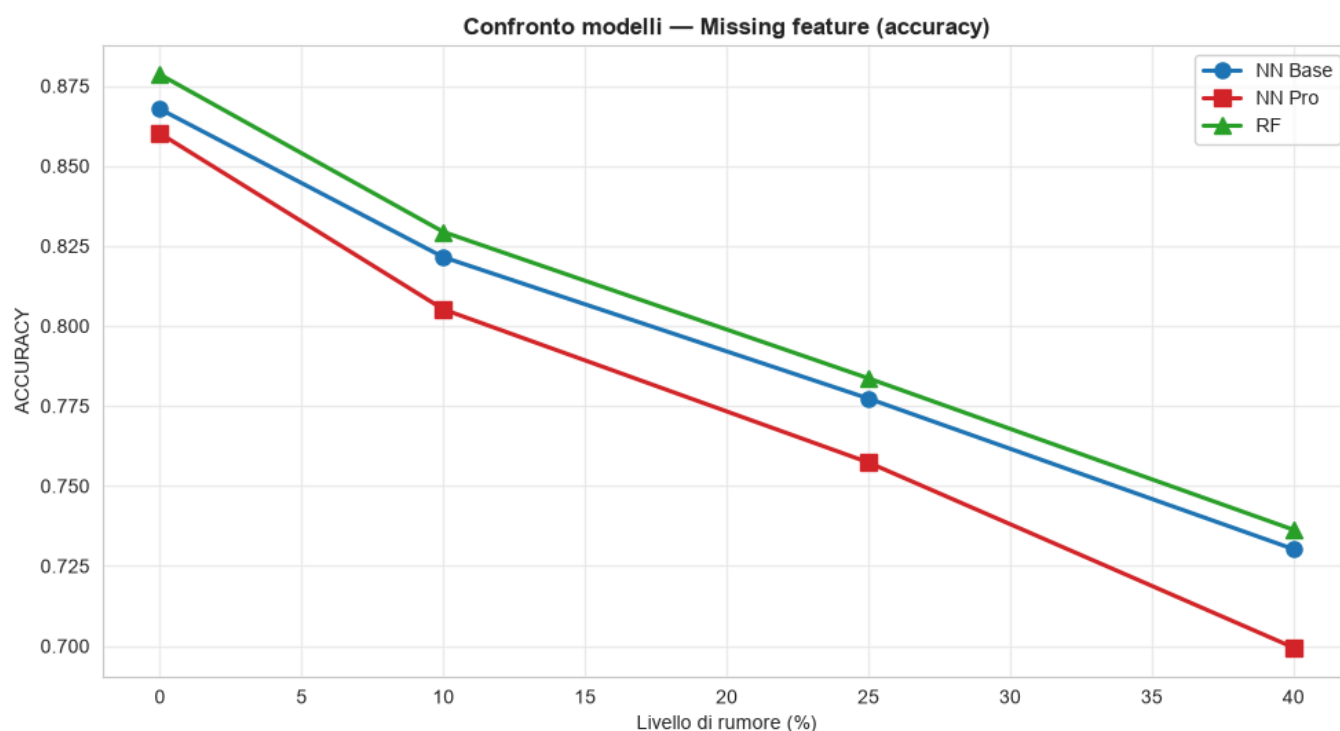
Perché. Due ragioni. (1) Il RF **non guarda la magnitudine**: i suoi split sono soglie basate sull'*ordinamento*, e un valore a $+5\sigma$ sta «oltre la soglia» esattamente come uno a $+2\sigma$ → l'albero non se ne accorge. (2) Per le reti, i dati erano già standardizzati e i veri outlier del dataset (le code pesanti di §2.3) erano già nel training: aggiungerne pochi altri a $\pm 5\sigma$ tocca poche celle. La **winsorizzazione** li riporta nel range e cancella il piccolo danno residuo.

4.F — Missing Feature (sensore guasto: feature azzerate) · *colpisce dove fa più male*

Cosa fa. Azzerare *intere* feature (sensori guasti). Non c'è un «livello %» continuo: i tre livelli mappano sulla rimozione delle feature **più importanti** (per importanza RF) — **10% → top-1 (fAlpha)**, **25% → top-3 (fAlpha, fLength, fWidth)**, **40% → top-5 (+fSize, fM3Long)**. Rimozione coerente su train/val/test.

Modello	Metrica	0%	10%	25%	40%	$\Delta(0 \rightarrow 40)$
NN base	Accuracy	0.8678	0.8215	0.7773	0.7303	-0.1375
	AUC	0.9214	0.8675	0.8007	0.7413	-0.1801
	Recall hadron (0)	0.77	0.65	0.54	0.43	-0.34

Modello	Metrica	0%	10%	25%	40%	$\Delta(0 \rightarrow 40)$
NN pro	Accuracy	0.8601	0.8052	0.7574	0.6995	-0.1606
	AUC	0.9238	0.8716	0.7991	0.7347	-0.1891
	Recall hadron (0)	0.80	0.72	0.62	0.55	-0.25
RF	Accuracy	0.8785	0.8294	0.7836	0.7363	-0.1422
	AUC	0.9300	0.8788	0.8195	0.7494	-0.1806
	Recall hadron (0)	0.77	0.66	0.57	0.46	-0.31



Confronto dei tre modelli su accuracy al crescere delle feature rimosse: crollo ripido già dal primo livello.

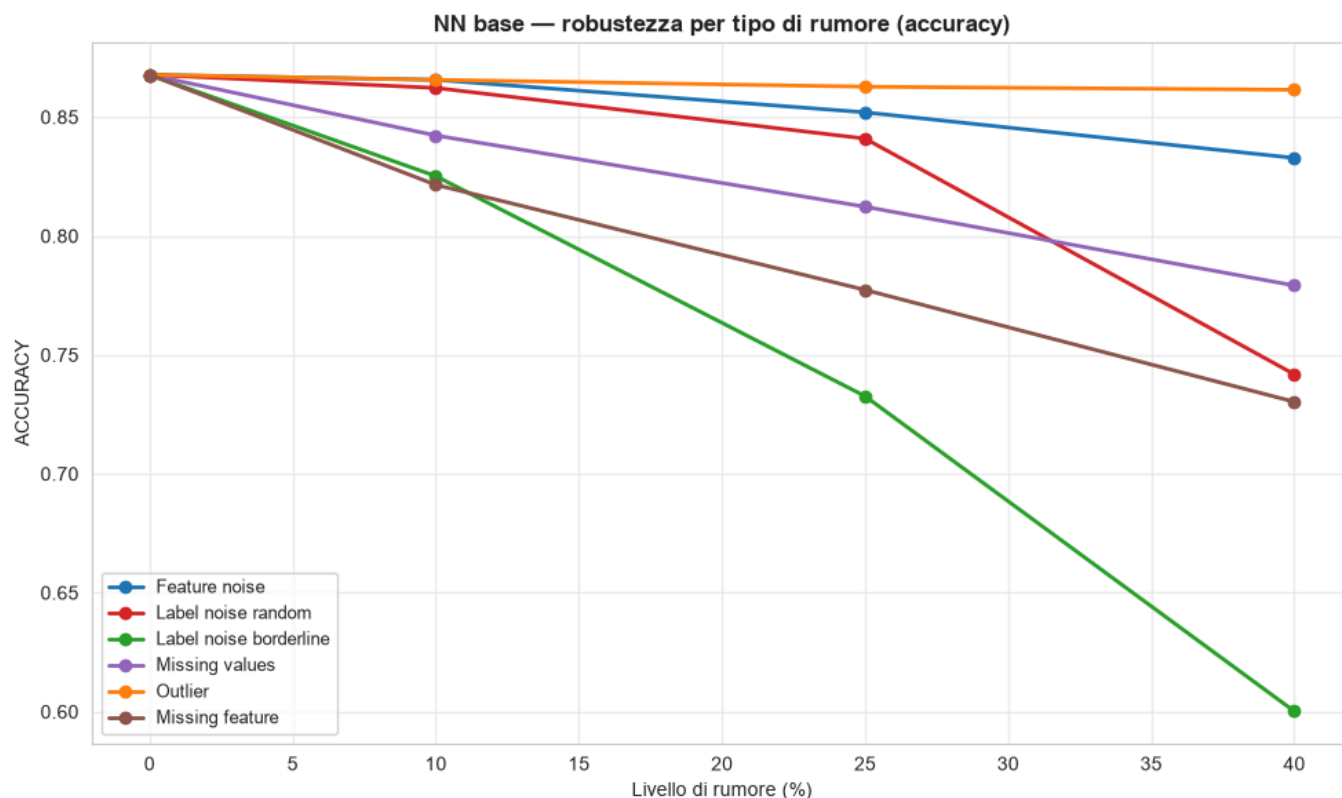
Lettura. Degrado **ripido** ($\Delta_{acc} -0.14 \div -0.18$ al 40%), e — punto chiave — **già il primo livello (sola fAlpha) costa ~5 punti** a tutti. Il **RF resiste meglio** (0.7363 @40%), la **NN Pro un po' peggio** (0.6995). Il recall hadron è quello che collassa di più: NN Base @40% scende a **0.43** (meno di un hadron su due viene riconosciuto).

Perché. Qui non c'è ridondanza che tenga: rimuovere **fAlpha** per prima significa togliere **proprio la variabile indipendente e più discriminante** (§2.1–2.2), quella che nessun'altra può rimpiazzare. È l'altra faccia del feature noise: là il danno era *diffuso e mediabile*, qui è *concentrato sulla colonna che conta*. Il RF regge un filo meglio perché ridistribuisce sulle feature residue (gli alberi ripartono dalle correlazioni **fLength/fWidth/fSize**); la NN Pro è penalizzata perché, quando il segnale vero scarseggia, il **class_weight** la spinge a «coprire» di più la classe minoritaria, aumentando gli errori sui gamma senza salvare abbastanza hadron.

5. Sintesi trasversale — i grafici di confronto e di sintesi

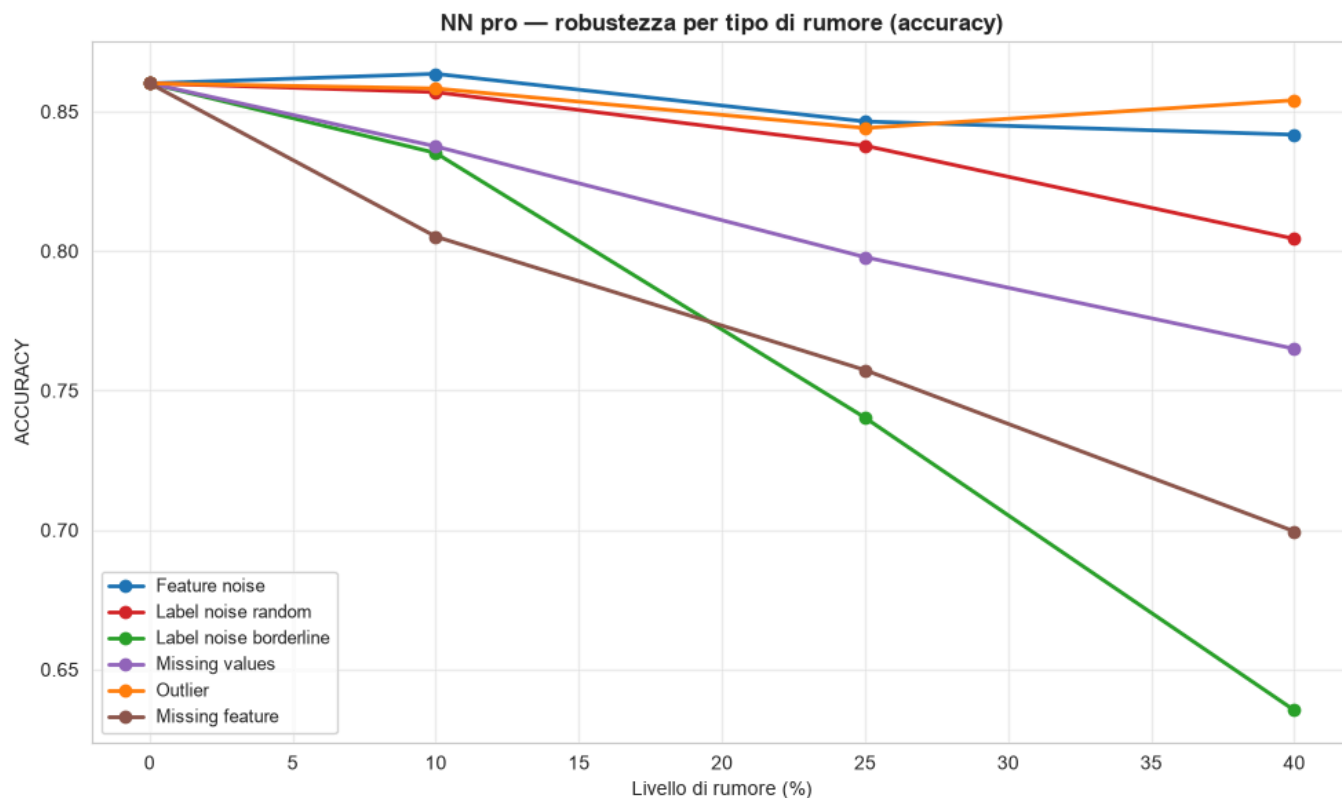
Questa è la parte che **collega** tutto. I grafici aggregati mettono, per ogni modello, le 6 curve di rumore nello stesso piano.

5.1 Profilo di robustezza per modello



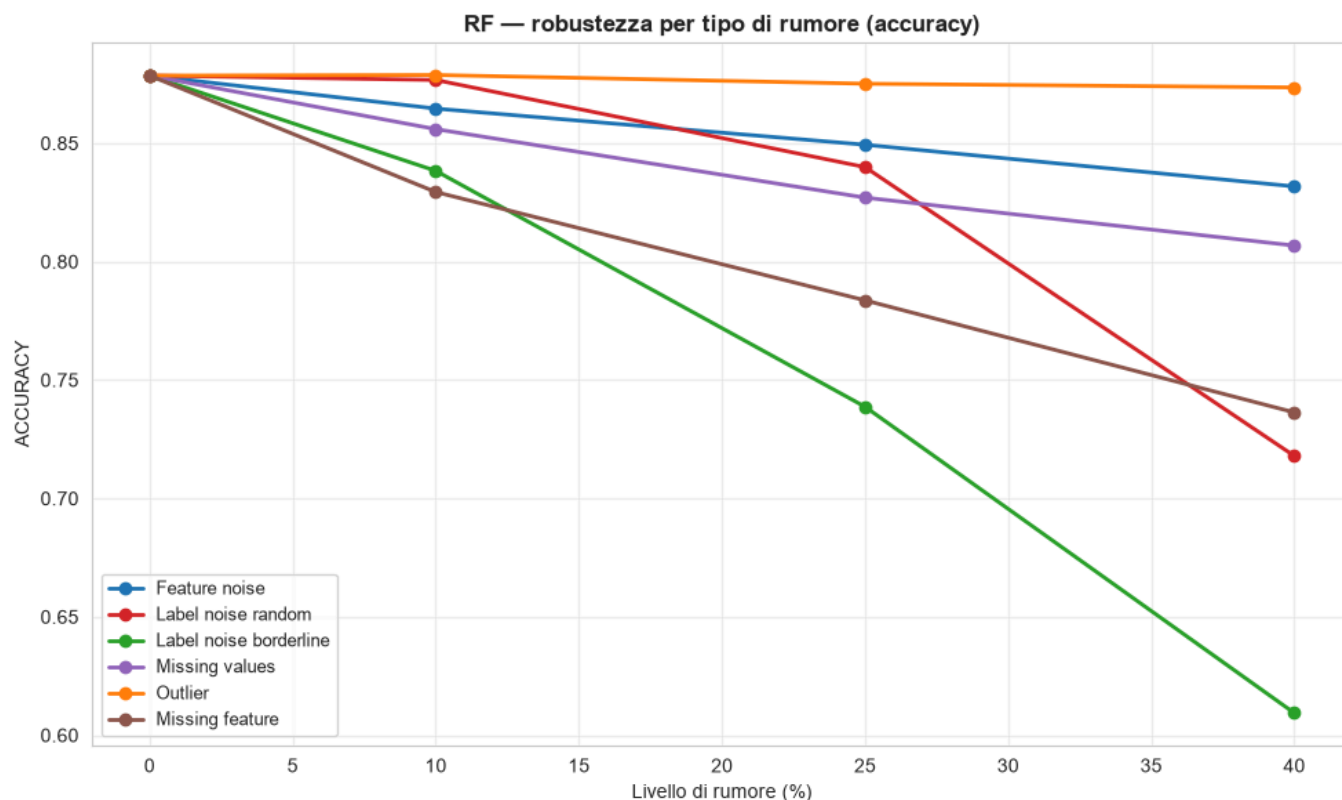
NN Base: accuracy contro tutti i 6 rumori. Ventaglio largo, dall outlier (piatto) al borderline (crollo a 0.60).

NN Base mostra il **ventaglio più aperto**: in alto e piatte le curve outlier e feature noise, in basso il crollo del borderline. È il modello che *amplifica* le differenze fra rumori perché non ha difese: fa esattamente ciò che i dati gli dettano.



NN Pro: stesso piano. Curve più compatte e alte; la regolarizzazione comprime soprattutto il label noise random.

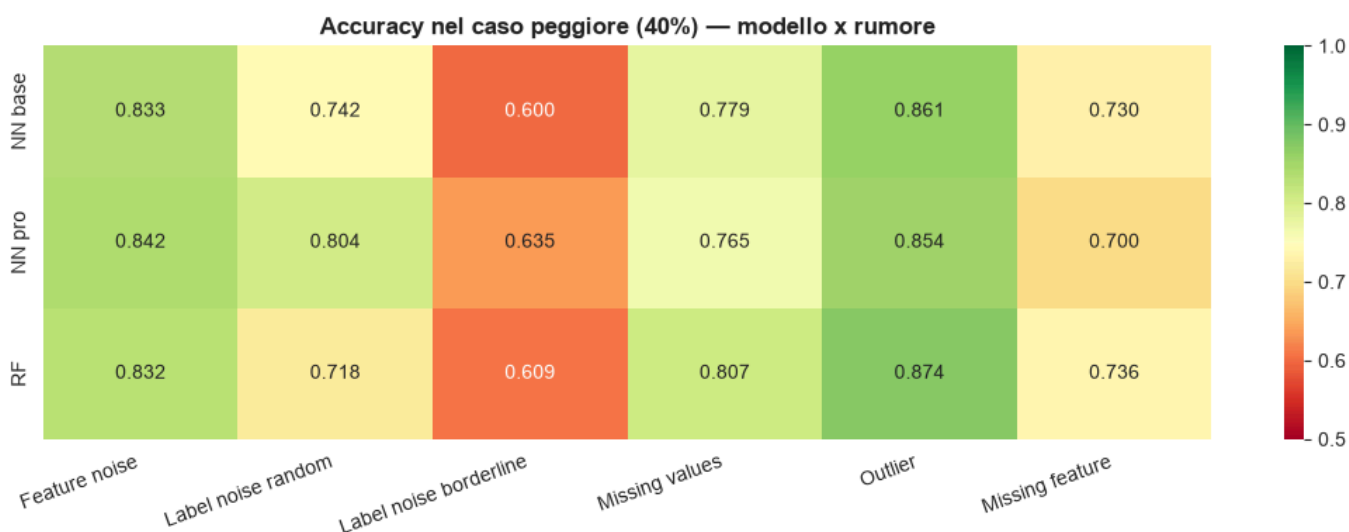
NN Pro «comprime» il ventaglio verso l'alto: la differenza più vistosa rispetto alla Base è la curva **label noise random**, che resta molto più alta (0.804 vs 0.742). È la prova visiva che la regolarizzazione agisce *selettivamente* dove serve — sul rumore che si combatte non memorizzando.



RF: stesso piano. Outlier e missing values altissimi, ma il label noise random precipita più che negli altri.

RF tiene altissime outlier e missing values, ma la curva **label noise random** precipita più che negli altri modelli ($\Delta -0.16$): si rivede il crossover di §4.B. Conferma del principio: il RF è forte sull'*integrità dei dati*, debole sulla *memorizzazione delle etichette*.

5.2 Heatmap del caso peggiore (40%)



Accuracy al 40% per modello x rumore. Colonna *borderline* tutta rossa; colonne *outlier*/*feature noise* verdi.

Si legge a colpo d'occhio la dicotomia del TL;DR. **Per colonna** (chi vince ogni rumore al 40%): NN Pro vince *feature noise*, *label random*, *label borderline*; RF vince *missing values*, *outlier*, *missing feature*; NN Base non vince mai. **Per colore**: la colonna *borderline* è la banda rossa (tutti in difficoltà), *outlier* e *feature noise* sono verdi (tutti tranquilli).

5.3 Quanto si perde: la degradazione 0 → 40%

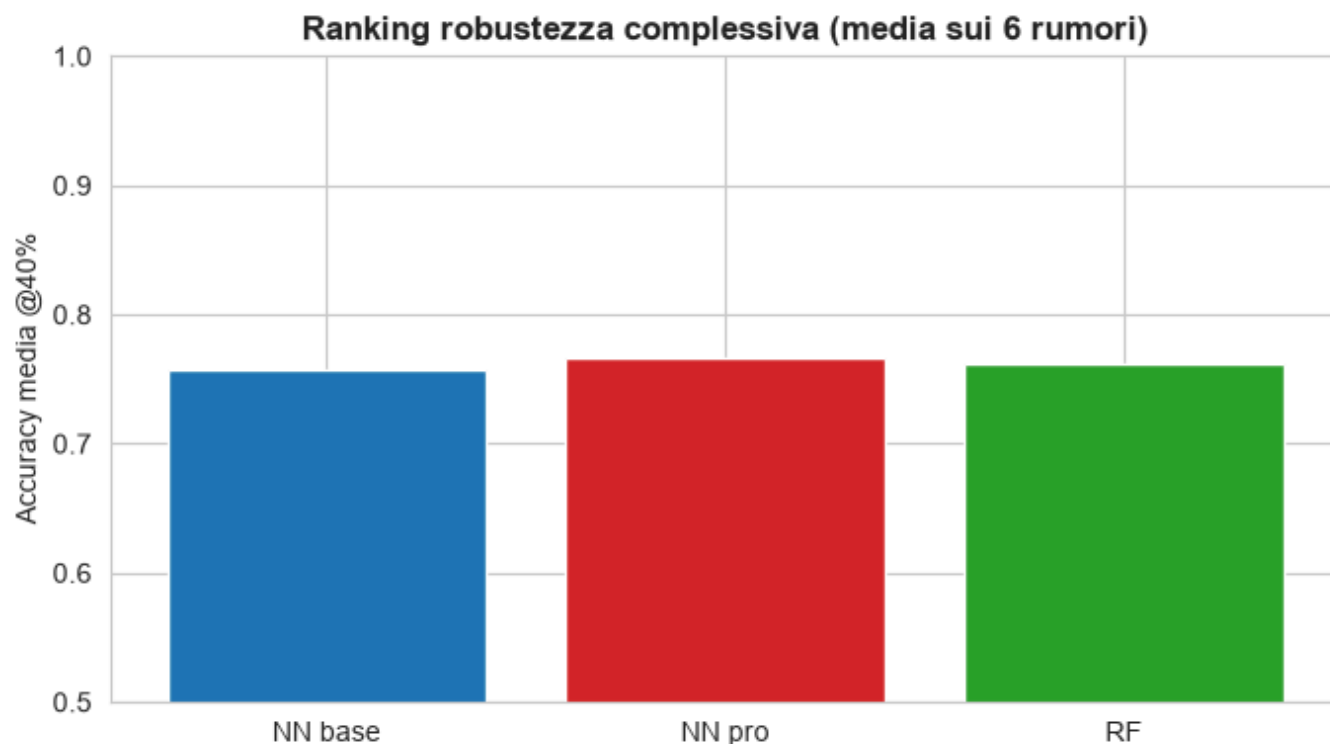


Calo di accuracy 0 → 40% per rumore e modello. Barra NN Pro del *label-noise-random* minuscola; quella RF la più alta.

Il grafico rende plastico il risultato centrale: sul **label noise random** la barra della **NN Pro** è quasi **inesistente (−0.055)** mentre quella del **RF** è la più alta (−0.16). Sul **borderline** perdono tutti tanto

(~0.22–0.27). Su **outlier** le barre sono praticamente a zero. Sul **missing feature** è la **NN Pro** ad avere la barra più alta (−0.16): l'unico rumore in cui la regolarizzazione non aiuta, perché il problema non è la varianza ma l'assenza di segnale.

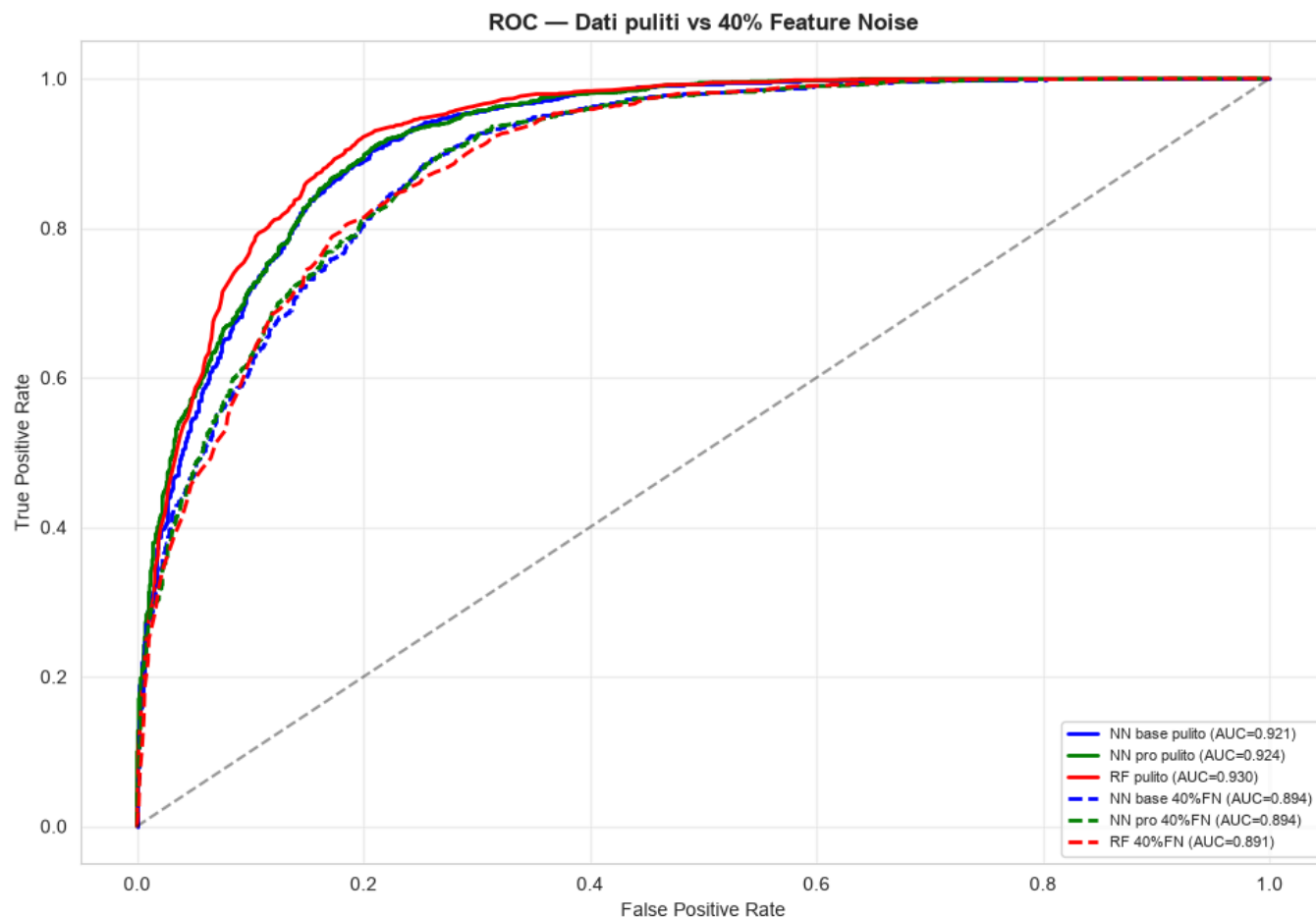
5.4 Ranking complessivo



Accuracy media @40% sui 6 rumori: le tre barre quasi identiche, NN Pro di pochissimo davanti.

Le tre barre sono **quasi identiche** (NN Pro 0.767 · RF 0.763 · NN Base 0.758). È il messaggio finale: **mediato su tutto, nessun modello domina**. La scelta giusta dipende da *quale* rumore ti aspetti — ed è esattamente ciò che le sezioni precedenti permettono di decidere.

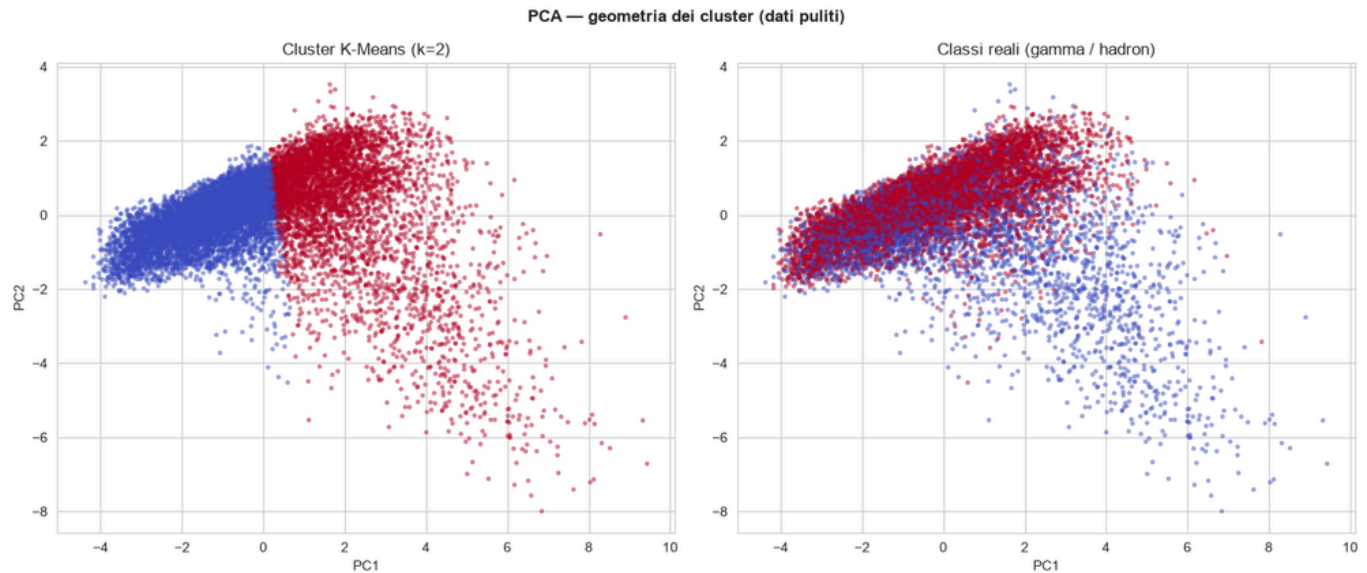
5.5 ROC — dati puliti vs 40% feature noise



Curve ROC: pieno = dati puliti, tratteggiato = 40% feature noise. Tutte scivolano da circa 0.92-0.93 a circa 0.89.

Le tre curve **piene** (puliti: RF 0.930 $\geq$ NN Pro 0.924 $\approx$ NN Base 0.921) sono incollate in alto a sinistra. Col 40% di feature noise le **tratteggiate** scendono compatte a ~ 0.89 , con il RF che perde un filo di più (-0.039 vs $-0.027/-0.030$ delle reti, coerente con §4.A). Il punto da portare a casa: anche al massimo feature noise le curve restano **molto sopra la diagonale** $\rightarrow$ la capacità di *ordinare* gli eventi resta buona; il feature noise erode il margine, non la sostanza.

6. Modello non supervisionato (K-Means) — perché *non* funziona, ed è interessante che non funzioni



PCA 2D: a sinistra i cluster K-Means (taglio netto su PC1), a destra le classi reali completamente mescolate.

Il K-Means (k=2) sui dati puliti dà metriche **bassissime**: ARI 0.006, AMI/NMI ~0.001, **purezza 0.648**, silhouette 0.290.

La PCA spiega perché in un colpo solo: a **sinistra** K-Means produce due cluster netti tagliando lungo **PC1** (la direzione di massima varianza); a **destra** le classi reali (gamma/hadron) sono **completamente sovrapposte** nello stesso piano. Cioè **la geometria naturale dei dati non coincide con le classi fisiche**: la separazione gamma/hadron non è la direzione di varianza dominante (è guidata soprattutto da **fAlpha**, che non è il primo asse principale). Una purezza di 0.648 = praticamente la frazione della classe maggioritaria → K-Means non fa meglio del «predico sempre gamma».

Due esperimenti chiudono il ragionamento e **collegano** la parte non supervisionata a quella supervisionata:

σ (feature noise)	ARI	AMI	silhouette	purezza
0.00	0.006	0.001	0.290	0.648
0.25	0.008	0.002	0.272	0.648
0.50	0.009	0.002	0.234	0.648
1.00	0.006	0.002	0.164	0.648
1.50	0.005	0.002	0.125	0.648
2.00	0.003	0.001	0.102	0.648

- **Feature noise** → la **silhouette crolla** (0.290 → 0.102): i cluster diventano sempre meno compatti/separati. Ma ARI e purezza **non si muovono**, perché non erano comunque allineati alle classi. Il feature noise *peggiora la geometria*, non «scollega» qualcosa che non era collegato.
- **Label noise** → **effetto zero**. ARI fra il clustering pulito e quello con label noise al 40% = **1.000**: ovvio, perché **K-Means usa solo le feature e ignora del tutto le etichette**. È il complemento perfetto del supervisionato: lì il label noise era *il problema*, qui è invisibile.

Morale: su MAGIC la **supervisione è indispensabile**. L'informazione di classe non è scoperta dalla struttura non supervisionata; sta in **fAlpha** e va «indicata» al modello con le etichette.

7. Nota onesta sugli Intervalli di Confidenza

Il notebook contiene una **sezione di markdown** dedicata agli intervalli di confidenza ($N=7$ seed, IC al $95\% = \bar{x} \pm 1.96 \cdot \sigma / \sqrt{N}$) su feature noise e label noise, e le conclusioni vi fanno riferimento. **Però in questa esecuzione la cella di calcolo/grafico degli IC non è presente** (dopo la ROC si passa direttamente al K-Means): quindi nei risultati **non ci sono numeri né figure** sugli intervalli di confidenza. Lo segnalo perché è l'unico punto in cui «conclusione attesa» e «output prodotto» non combaciano.

Cosa *mostrerebbe* quell'analisi, e perché sarebbe coerente con il resto: se i cali visti (specie il crollo da label noise) avessero IC **stretti**, sarebbero un **segnale reale dei dati sporchi** e non rumore da inizializzazione random. Visto che i trend qui sono **monotoni e netti** (es. NN Base label random $0.868 \rightarrow 0.862 \rightarrow 0.841 \rightarrow 0.742$, sempre in discesa), è quasi certo che gli IC risulterebbero stretti rispetto all'entità del calo. La cella si aggiunge in poche righe (ciclo su 7 seed, **np.mean/np.std**, barre d'errore).

8. Conclusioni operative

Per tipo di rumore (cosa fare se lo incontri):

- **Outlier** → quasi irrilevante; se vuoi pulire, **winsorizza** (taglio code). Il RF non ha bisogno di nulla.
- **Feature noise** → degrado dolce e mediabile; **regolarizza la rete** (la NN Pro è la più piatta). Sfrutta la ridondanza fra feature.
- **Missing values** → **imputa con la mediana** (feature asimmetriche); il RF tollera meglio l'imputazione grazie alla ridondanza.
- **Missing feature** → l'antidoto è **a monte**: proteggi/ridonda il sensore di **fAlpha**. È la feature che, persa, costa di più; nessun modello la rimpiazza.
- **Label noise random** → **regolarizza e usa early stopping** (NN Pro +6 punti sulla Base al 40%). Attenzione: il **RF non è la scelta sicura** ad alto rumore — i suoi alberi profondi memorizzano gli errori. Volendo tenere il RF, andrebbe **potato** (**max_depth**, **min_samples_leaf**).
- **Label noise borderline** → il più pericoloso e subdolo; le difese standard limitano i danni ma non li annullano, perché il confine viene *strutturalmente* falsato. Qui conta la **qualità dell'annotazione vicino al confine** più di qualsiasi modello.

Per modello (riassunto del carattere):

- **NN Base** = termometro. Senza difese, *amplifica* ogni rumore; utile proprio per *vedere* gli effetti (overfitting da label noise in primis).
- **NN Pro** = lo specialista del **segnale corrotto** (feature/label noise): dropout + L2 + early stopping = freno alla memorizzazione. Più equilibrata fra le classi (class_weight). Debole solo quando il segnale *manca* (missing feature).
- **RF** = lo specialista dell'**integrità dei dati** (outlier, missing values, missing feature): split robusti + ridondanza. Tallone d'Achille: la **memorizzazione delle etichette** ad alto label noise.

Il filo conduttore di tutto lo studio: la robustezza non è una proprietà del modello «in assoluto», ma dell'**accoppiamento fra il meccanismo del rumore e l'architettura**. Rumore *diffuso* e *a media nulla*

(feature, outlier) → si media via, vincono i robusti per costruzione. Rumore che *cambia il bersaglio* (label) → vince chi non memorizza. Rumore che *toglie il segnale* (missing feature) → non c'è modello che tenga, conta solo quanto era informativa la colonna persa — e su MAGIC quella colonna ha un nome: **fAlpha**.